

PROGRAMMATION FONCTIONNELLE 2 : TD10 PRÉDICATS INDUCTIFS, ENCORE

avril 2026 — Pierre Rousselin

On rappelle qu'on a défini sur `nat` le prédicat inductif `le` et la fonction booléenne `leb` de la façon suivante :

```
Inductive le (n : nat) : nat -> Prop :=
  le_n : le n n | le_S : forall m : nat, le n m -> le n S m.
Notation "n <= m" := (le n m).
Fixpoint leb n m :=
  match n with
  | 0 => true
  | S n' => match m with
    | 0 => false
    | S m' => leb n' m'
  end
end.
Notation "n <=? m" := (leb n m).
```

Par ailleurs, on a les lemmes de réflexion :

```
Lemma le_leb (n m : nat) : n <= m -> n <=? m = true. Proof. (* ... *) Qed.
Lemma leb_le (n m : nat) : n <=? m = true -> n <= m. Proof. (* ... *) Qed.
```

Le prédicat correspondant intuitivement à « être une liste triée d'entiers naturels » est :

```
Inductive sorted : list nat -> Prop :=
  | nil_sorted : sorted []
  | one_sorted (x : nat) : sorted [x]
  | cons_sorted (h1 h2 : nat) (t : list nat) :
    sorted (h2 :: t) -> h1 <= h2 -> sorted (h1 :: h2 :: t).
```

Exercice 1 :

1. Réécrire la définition de `sorted` sous la forme de règles d'inférences.
2. Donner l'arbre de dérivation pour prouver `sorted [10; 42; 93]`.
3. Donner la définition en Rocq de la fonction

```
(** retourne la liste [start; S (start); ... (start + len - 1)] *)
Fixpoint list_of_nats (start len : nat) := (* à compléter *)
```

4. Donner la preuve papier et/ou Rocq du lemme suivant :

```
Lemma sorted_list_of_nat (start len : nat) : sorted (list_of_nats start len).
```

Faire attention à la généralité de l'hypothèse d'induction (il est instructif de se lancer dans une induction non généralisée et de voir à quel endroit ça coince).

--- * ---

Correction de l'exercice 1 :

1. Pas le temps.
2. Pas le temps.

3.

```
Fixpoint list_of_nats (start len : nat) :=
  match len with
  | 0 => []
  | S q => start :: (list_of_nats (S start) q)
end.
```

4. **Lemma** sorted_list_of_nat (start len : nat) : sorted (list_of_nats start len).

Proof.
`induction len as [| len' IH] in start |- *.
 - simpl. exact nil_sorted.
 - destruct len' as [| len''].
 + exact (one_sorted _).
 + simpl.
 apply cons_sorted; [| lia].
 simpl in IH.
 exact (IH (S start)).`
Qed.

--- * ---

Exercice 2 : Permutation

Le prédicat inductif correspondant intuitivement à « deux listes sont permutation l'une de l'autre » est défini de la façon suivante :

Inductive Permutation {A : Type} : list A -> list A -> Prop :=
 | perm_nil : Permutation [] []
 | perm_skip (x : A) (l l' : list A) :
 Permutation l l' -> Permutation (x :: l) (x :: l')
 | perm_swap (x y : A) (l : list A) :
 Permutation (y :: x :: l) (x :: y :: l)
 | perm_trans (l l' l'' : list A) :
 Permutation l l' -> Permutation l' l'' -> Permutation l l''.

1. Donner la preuve en Rocq de

Lemma Permutation_exple1 : Permutation [1; 42; 93] [1; 93; 42].

2. Idem pour

Lemma Permutation_exple2 : Permutation [42; 93; 2; 1] [93; 42; 1; 2].

3. Prouver, en Rocq et/ou sur papier, par induction sur la preuve de Permutation l1 l2 :

Lemma Permutation_length {A : Type} (l1 l2 : list A) :
 Permutation l1 l2 -> length l1 = length l2.

--- * ---

Correction de l'exercice 2 :

1. **Lemma** Permutation_exple1 : Permutation [1; 42; 93] [1; 93; 42].

Proof.
`apply perm_skip; exact (perm_swap _ _ _).
Qed.`

2. **Lemma** Permutation_exple2 : Permutation [42; 93; 2; 1] [93; 42; 1; 2].

Proof.
`apply (perm_trans _ [42; 93; 1; 2]).
 - apply perm_skip, perm_skip; exact (perm_swap _ _ _).
 - exact (perm_swap _ _ _).
Qed.`

3. **Lemma** Permutation_length {A : Type} (l1 l2 : list A) :
 Permutation l1 l2 -> length l1 = length l2.

Proof.
`intros H.
induction H as [| x l l' H IH | x y l | l l' l'' H1 IH1 H2 IH2].
 - reflexivity.
 - simpl; rewrite IH; reflexivity.
 - simpl; reflexivity.
 - rewrite IH1 -IH2; reflexivity.
Qed.`

--- * ---

Exercice 3 : Preuve par cas « trop destructive » sur sorted

Terminer :

- le nommage dans les cas de `destruct`,
- la preuve en Rocq, en remplaçant chaque fois `l` par la liste correspondante

dans la preuve suivante :

```
Lemma sorted_cons (h : nat) (t : list nat) :
  sorted (h :: t) -> sorted t.
Proof.
  remember (h :: t) as l eqn: def_l.
  intros H; destruct H as [...].
```

La tactique

```
remember (h :: t) as l eqn: def_l.
```

introduit dans le contexte l'équation

```
def_l : l = h :: t
```

et change toutes les autres occurrences `h :: t` du contexte et du but en `l`.

--- * ---

Correction de l'exercice 3 :

```
(** un destruct (avec remember) sur sorted : *)
Lemma sorted_cons (h : nat) (t : list nat) :
  sorted (h :: t) -> sorted t.
Proof.
  remember (h :: t) as l eqn: def_l.
  intros H; destruct H as [| x | h1 h2 t2 Hh2t2 Hh1h2].
  - (* l = [] : impossible *) discriminate def_l.
  - (* l = [x] donc t = [] *)
    injection def_l as _ Ht. rewrite -Ht. exact nil_sorted.
  - (* l = h1 :: h2 :: t2, donc t = h2 :: t2 et est déjà
      triée : *)
    injection def_l as _ Ht. rewrite -Ht. exact Hh2t2.
Qed.
```

--- * ---

Exercice 4 : Réflexion booléenne pour sorted

1. Donner la définition en Rocq de la fonction `sortedb : list nat -> bool` qui vaut `true` si et seulement si son argument est une liste triée dans l'ordre croissant.

2. Quelle est la preuve commune des deux lemmes suivants?

```
Lemma sortedb_cons_cons_leb (h h' : nat) (l : list nat) :
  h <= h' = true -> sortedb (h :: h' :: l) = sortedb (h' :: l).
Proof. (* .... *) Qed.
Lemma sortedb_cons_cons_nleb (h h' : nat) (l : list nat) :
  h <= h' = false -> sortedb (h :: h' :: l) = false.
Proof. (* .... *) Qed.
```

3. Donner la preuve en Rocq de

```
Lemma sortedb_sorted (l : list nat) : sortedb l = true -> sorted l.
```

4. Donner la preuve en Rocq de

```
Lemma sorted_sortedb (l : list nat) : sorted l -> sortedb l = true.
```

5. En déduire :

```
Lemma sorted_decidable (l : list nat) : sorted l \ / ~ sorted l.
```

--- * ---

Correction de l'exercice 4 :

1. `Fixpoint sortedb (l : list nat) : bool :=`

```

  match l with
  | [] => true
  | h :: t => match t with
    | [] => true
    | h' :: t' => (h <=? h') && sortedb t
  end
end.
```
2. `Proof. intros H; simpl; rewrite H; reflexivity. Qed.`
3. `Lemma sortedb_sorted (l : list nat) : sortedb l = true -> sorted l.`
`Proof.`

```

  induction l as [| h t IH].
  - intros _; exact nil_sorted.
  - destruct t as [| h' t'].
    + intros _; exact (one_sorted h).
    + destruct (h <=? h') eqn:E.
      * rewrite sortedb_cons_cons_leb; [exact E |].
        intros Hsb.
        apply cons_sorted; [apply IH | apply leb_le]; assumption.
      * rewrite sortedb_cons_cons_nleb; [exact E |].
        intros H; discriminate H.
Qed.
```
4. `Lemma sorted_sortedb (l : list nat) : sorted l -> sortedb l = true.`
`Proof.`

```

  intros H.
  induction H as [| x | h1 h2 t H IH Hh1h2].
  - reflexivity.
  - reflexivity.
  - rewrite sortedb_cons_cons_leb.
    + apply le_leb; exact Hh1h2.
    + exact IH.
Qed.
```
5. `Lemma sorted_decidable (l : list nat) : sorted l \ / ~ sorted l.`
`Proof.`

```

  destruct (sortedb l) eqn:E.
  - left; apply sortedb_sorted; exact E.
  - right; intros H; apply sorted_sortedb in H; rewrite E in H.
    discriminate H.
Qed.
```

--- * ---